第三部分

活跃性、性能与测试

第 10 章

避免活跃性危险

在安全性与活跃性之间通常存在着某种制衡。我们使用加锁机制来确保线程安全，但如果过度地使用加锁，则可能导致锁顺序死锁（Lock-Ordering Deadlock）。同样，我们使用线程池和信号量来限制对资源的使用，但这些被限制的行为可能会导致资源死锁（Resource Deadlock）。Java 应用程序无法从死锁中恢复过来，因此在设计时一定要排除那些可能导致死锁出现的条件。本章将介绍一些导致活跃性故障的原因，以及如何避免它们。

10.1 死锁

经典的“哲学家进餐”问题很好地描述了死锁状况。5 个哲学家去吃中餐，坐在一张圆桌旁。他们有 5 根筷子（而不是 5 双），并且每两个人中间放一根筷子。哲学家们时而思考，时而进餐。每个人都需要一双筷子才能吃到东西，并在吃完后将筷子放回原处继续思考。有些筷子管理算法能够使每个人都能相对及时地吃到东西（例如一个饥饿的哲学家会尝试获得两根邻近的筷子，但如果其中一根正在被另一个哲学家使用，那么他将放弃已经得到的那根筷子，并等待几分钟之后再次尝试），但有些算法却可能导致一些或者所有哲学家都“饿死”（每个人都立即抓住自己左边的筷子，然后等待自己右边的筷子空出来，但同时又不放下已经拿到的筷子）。后一种情况将产生死锁：每个人都拥有其他人需要的资源，同时又等待其他人已经拥有的资源，并且每个人在获得所有需要的资源之前都不会放弃已经拥有的资源。

当一个线程永远地持有一个锁，并且其他线程都尝试获得这个锁时，那么它们将永远被阻塞。在线程 A 持有锁 L 并想获得锁 M 的同时，线程 B 持有锁 M 并尝试获得锁 L，那么